ADR-001 · DÉCISION D'ARCHITECTURE

Choix du langage et de la plateforme serveur

Étude comparative et justification pour le backend de la table de jeu de rôle virtuelle (sujet EpiQuest, Programme Grande École, 5e année)

Statut	**Accepté**	**Date**	28 octobre 2026
Auteur	Quentin, responsable backend	**Périmètre**	Serveur / API, moteur de règles, synchronisation temps réel
Décision	**C# sur .NET 10 (LTS)** : ASP.NET Core, SignalR, EF Core, PostgreSQL	**Révision**	À réexaminer si l'un des déclencheurs de la section 9 survient

01 Résumé de la décision

> Le serveur est développé en **C# sur .NET 10 (LTS)**, avec **ASP.NET Core** pour l'API, **SignalR** pour la synchronisation des deux vues, **EF Core** et **PostgreSQL** pour la persistance. Le moteur de règles est une bibliothèque C# pure, sans dépendance d'infrastructure, exécutée uniquement côté serveur. Le web et le mobile l'utilisent à travers l'API, ce qui garantit un moteur unique.

Six langages ont été comparés sur sept critères pondérés, tirés des exigences du sujet. C# / .NET obtient le meilleur score (4,55 / 5). Il domine sur les deux critères les plus lourds : la modélisation du moteur de règles et la synchronisation temps réel avec filtrage par audience. TypeScript est le seul concurrent doté d'un avantage structurel (partage du moteur avec un front React), mais l'absence d'expérience du responsable backend dans ce langage, sur un projet de trois mois, l'emporte sur ce gain.

02 Contexte et contraintes du sujet

Le sujet laisse la stack libre, à condition de la justifier dans la documentation technique (bloc 01-B). Les exigences qui pèsent directement sur le serveur sont les suivantes.

Exigence	Réf.	Conséquence pour le langage serveur
Règles décrites comme des données, interprétées par un moteur (sorts, capacités, monstres)	02-C, A4	Modéliser des effets hétérogènes (attaque, sauvegarde, dégâts, soins, état, modificateur, effet libre), les sérialiser en JSON et les interpréter sans code spécifique par sort.
Le serveur fait autorité : dés tirés par un générateur sûr, actions de combat validées côté serveur	02-E, 03-B	Générateur cryptographique disponible en standard ; résolution des actions côté serveur uniquement.
Combat assisté : résultat affiché avant application, modifiable, annulable, historisé	02-H, A1	Journal de commandes ou d'événements, avec annulation par événement inverse ou rejeu.
Deux vues synchronisées en direct ; la vue joueurs ne reçoit jamais d'information cachée	02-I, 03-B	Temps réel avec groupes par audience ; projection filtrée par destinataire, calcul de la ligne de vue côté serveur.

Exigence	Réf.	Conséquence pour le langage serveur
Le mobile utilise le même moteur de règles que le web	02-K	Moteur partagé en bibliothèque, ou moteur unique côté serveur exposé par l'API.
Comptes, permissions, mots de passe hachés, sessions sécurisées, Top 10 OWASP	03-B	Écosystème d'authentification et de persistance mature, pour ne pas réinventer ces briques.
Installation en une commande, tests en une commande, CI/CD, hébergement gratuit	03-B	Bon support Docker, migrations de schéma automatisées, outillage de test et d'intégration continue.
Chaque membre doit pouvoir expliquer n'importe quelle partie du code	03-D	Langage maîtrisé ou rapidement lisible par toute l'équipe.

Contexte de l'équipe. Groupe de trois étudiants, du 19 octobre 2026 à fin janvier 2027. Le responsable backend maîtrise C# / .NET et Go, et n'a pas d'expérience en TypeScript.

03 Critères d'évaluation et pondération

Les poids reflètent la difficulté et le poids de chaque sujet dans l'évaluation. Le moteur de règles et la synchronisation sont au cœur du sujet (bloc 01-A) ; les exigences de qualité comptent autant que les fonctionnalités (bloc 03).

#	Critère	Poids	Ce qui est évalué
C1	**Modélisation du moteur de règles**	**25 %**	Types somme ou hiérarchies scellées, pattern matching exhaustif, sérialisation JSON polymorphe, immutabilité.
C2	**Temps réel et filtrage par audience**	**20 %**	Groupes de diffusion, reconnexion, client navigateur, facilité à envoyer une projection différente par vue.
C3	**Sécurité, authentification, persistance**	**15 %**	Hachage, sessions, autorisations, ORM et migrations, protections OWASP disponibles sans les écrire.
C4	**Maîtrise de l'équipe backend**	**15 %**	Productivité dès la première semaine ; capacité à expliquer le code en soutenance.
C5	**Partage de code avec les clients**	**10 %**	Possibilité de réutiliser le moteur ou les types côté web et mobile.
C6	**Outillage, CI/CD, déploiement**	**10 %**	Docker, tests en une commande, GitHub Actions, offres d'hébergement gratuites.
C7	**Lisibilité et simplicité**	**5 %**	Facilité de relecture en pull request par un coéquipier qui n'a pas écrit le code.

Notation de 1 (inadapté) à 5 (excellent). Le score final est la moyenne pondérée.

04 Alternatives étudiées

4.1 C# / .NET (ASP.NET Core, SignalR, EF Core)

- **Moteur de règles.** Records immuables, pattern matching avec avertissement en cas de cas oublié, JSON polymorphe natif (`[JsonDerivedType]`). Le format des effets s'exprime directement dans le système de types.
- **Temps réel.** SignalR fournit les groupes (vue MJ, vue joueurs), la reconnexion et un client JavaScript officiel. Le hub envoie à chaque groupe sa projection filtrée.
- **Sécurité et données.** ASP.NET Identity (hachage, verrouillage, cookies sécurisés), politiques d'autorisation, protection anti-CSRF ; EF Core gère les migrations appliquées au démarrage.
- **Plateforme.** .NET 10 est une version LTS, supportée jusqu'au 14 novembre 2028.
- **Limites.** Plus de cérémonie que Go ; images Docker plus lourdes (atténué par les images *chiseled*) ; partage du moteur avec le mobile seulement si celui-ci est en .NET MAUI.

4.2 Go

- **Points forts.** Concurrence native (goroutines, canaux) idéale pour la diffusion temps réel ; binaire unique et image Docker minimale ; langage très lisible ; `crypto/rand` en standard. Maîtrisé par le responsable backend.
- **Limites.** Pas de types somme : chaque effet devient une interface, un switch sur un champ `type` et une désérialisation polymorphe écrite à la main, sans vérification d'exhaustivité. Groupes, reconnexion et authentification sont à assembler soi-même. Aucun partage possible avec le web ou le mobile.
- **Bilan.** Excellent pour un service de diffusion, moins adapté au cœur du sujet, qui est de la modélisation de domaine plus que du débit.

4.3 TypeScript (Node.js ou Bun)

- **Avantage structurel.** Si le web est en React et le mobile en React Native, un monorepo permet un package `rules-engine` unique utilisé partout, et des schémas Zod partagés pour valider l'import du SRD, le format des effets et l'API. C'est le seul argument capable de rivaliser avec la maîtrise d'un autre langage.
- **Limites.** Aucune expérience du responsable backend dans ce langage, sur un projet de trois mois où chaque membre doit pouvoir tout expliquer. Exécution mono-thread pour un calcul de ligne de vue CPU-bound (gérable, mais à surveiller). Sûreté des types limitée à la compilation sans validation à l'exécution. Vigilance accrue sur les dépendances.
- **Bilan.** Le meilleur choix pour une équipe déjà à l'aise en TypeScript, pas pour celle-ci.

4.4 Kotlin (Ktor, Kotlin Multiplatform)

- **Points forts.** Les `sealed class` et le `when` exhaustif sont idéaux pour les effets ; Kotlin Multiplatform permettrait de partager le moteur avec une application Android.
- **Limites.** Langage nouveau pour le responsable backend ; écosystème serveur (Ktor, Exposed) moins intégré qu'ASP.NET Core pour l'authentification. Le gain n'existe que si le mobile est en Kotlin.

4.5 Rust (Axum)

- **Points forts.** Enums algébriques parfaits pour le moteur, performances maximales, compilation possible du moteur en WebAssembly pour le navigateur.
- **Limites.** Courbe d'apprentissage trop forte pour trois mois (possession, durées de vie, async) ; relecture difficile pour les coéquipiers, ce qui va à l'encontre de l'exigence d'explicabilité.

4.6 Elixir (Phoenix)

- **Points forts.** Phoenix Channels et Presence sont parmi les meilleures briques temps réel existantes ; tolérance aux pannes de la machine virtuelle BEAM.
- **Limites.** Langage et paradigme nouveaux ; typage dynamique peu adapté à un format d'effets riche ; aucun partage avec le web ou le mobile.

05 Matrice de décision

Langage	C1 25 %	C2 20 %	C3 15 %	C4 15 %	C5 10 %	C6 10 %	C7 5 %	Score
C# / .NET	5	5	5	5	2	4	4	**4,55**
TypeScript	4	4	3	1	5	4	4	**3,50**
Go	2	4	3	5	1	5	5	**3,35**
Kotlin	5	3	3	1	3	4	3	**3,30**
Rust	5	3	3	1	2	4	1	**3,10**
Elixir	3	5	3	1	1	3	2	**2,85**

C1 moteur de règles · C2 temps réel et filtrage · C3 sécurité et persistance · C4 maîtrise de l'équipe · C5 partage avec les clients · C6 outillage et CI/CD · C7 lisibilité. Vert : 4-5, jaune : 3, rouge : 1-2.

Lecture de la matrice.

- C# / .NET arrive en tête avec un écart net (4,55 contre 3,50 pour TypeScript, deuxième). Son seul point faible, le partage de code avec les clients, porte sur un critère secondaire dans ce projet, puisque le serveur doit de toute façon tout revalider.
- **Analyse de sensibilité.** Même en donnant à TypeScript la note maximale en maîtrise (5 au lieu de 1), son score monte à 4,10, toujours sous celui de C#. Le choix ne dépend donc pas uniquement de la familiarité du responsable backend.
- Go reste un repli crédible grâce à la maîtrise de l'équipe, mais sa faiblesse sur C1 pèserait sur la partie la plus complexe du projet.

06 Décision

Brique	Choix retenu	Raison principale
Runtime	**.NET 10 (LTS)**	Supporté jusqu'au 14 novembre 2028. .NET 8 et .NET 9 arrivent en fin de support le 10 novembre 2026, pendant le projet ; .NET 11 sera une version à support court (STS).
API	**ASP.NET Core** (Minimal APIs)	OpenAPI généré nativement, validation, autorisations par politique, problem details standardisés.
Temps réel	**SignalR**	Groupes par audience (MJ, joueurs), reconnexion automatique, client JavaScript officiel.
Persistance	**EF Core** + **PostgreSQL**	Migrations versionnées, colonnes JSONB pour stocker les définitions de règles importées du SRD.
Authentification	**ASP.NET Identity**	Hachage des mots de passe, verrouillage, cookies sécurisés, sans réécrire ces briques sensibles.
Aléatoire	**RandomNumberGenerator**	Générateur cryptographique standard, côté serveur uniquement, pour des dés non truquables.

Brique	Choix retenu	Raison principale
Tests	**xUnit** + **Testcontainers**	Tests du moteur sans infrastructure ; tests d'intégration sur une vraie base PostgreSQL éphémère.

07 Conséquences

7.1 Le moteur de règles devient un moteur serveur unique

Le front web sera vraisemblablement en TypeScript : le moteur C# ne peut donc pas y être partagé tel quel. Il est exécuté côté serveur et exposé par l'API. Ce choix est cohérent avec le sujet : le serveur fait déjà autorité sur les dés et le combat (03-B), le mobile utilise bien « le même moteur » en appelant la même API (02-K), et aucun mode hors ligne n'est demandé.

Pour que les clients restent simples, le moteur ne se contente pas de valider : il **pilote l'interface**. À chaque étape de l'assistant de création, un endpoint sans état renvoie l'état calculé, les erreurs et les choix encore disponibles.

```
POST /api/characters/evaluate
→ {
    "derived":        { modificateurs, CA, PV, sauvegardes, emplacements… },
    "errors":         [ prérequis non remplis, trop de compétences… ],
    "pendingChoices": [ compétences restantes, sorts autorisés, sous-classes… ]
  }
```

7.2 Le contrat d'API devient le point de partage

- **OpenAPI**, généré par ASP.NET Core, sert à générer le client TypeScript du web et le client du mobile.
- **Messages SignalR** typés et documentés, pour que serveur et clients ne divergent pas.
- **JSON Schema** du format des effets, généré depuis les types C#, qui documente le format de description des règles exigé dans la documentation technique.

7.3 Le format des effets s'écrit dans le système de types

```
[JsonPolymorphic(TypeDiscriminatorPropertyName = "type")]
[JsonDerivedType(typeof(DamageEffect),    "degats")]
[JsonDerivedType(typeof(HealEffect),      "soins")]
[JsonDerivedType(typeof(ConditionEffect), "etat")]
[JsonDerivedType(typeof(ModifierEffect),  "modificateur")]
[JsonDerivedType(typeof(FreeEffect),      "libre")]
public abstract record Effect;

public sealed record DamageEffect(
    DiceExpression Dice, DamageType Type, bool HalfOnSave = false) : Effect;
public sealed record ConditionEffect(Condition Condition, Duration Duration) : Effect;
public sealed record FreeEffect(
    string Name, string Description, Trigger Trigger, Duration Duration) : Effect;
// HealEffect et ModifierEffect suivent le même modèle
```

L'interpréteur fait un `switch` sur le type d'effet ; le compilateur signale tout type d'effet non traité.

7.4 Bilan des conséquences

Positives	Négatives et mesures prises
Un seul moteur, testé une seule fois, impossible à contourner depuis un client.	Un aller-retour réseau par étape de l'assistant. *Mesure :* debounce côté client, endpoint sans état et rapide.
Moteur isolé dans une bibliothèque pure : tests de personnages de référence triviaux à écrire.	Quelques calculs purement visuels dupliqués côté client (gabarit survolé, distance affichée). *Mesure :* le serveur recalcule tout au moment de l'action ; duplication assumée et documentée.
Briques de sécurité éprouvées (Identity, autorisations, anti-CSRF) au lieu de code maison.	Image Docker plus lourde qu'en Go. *Mesure :* build multi-étapes et image runtime *chiseled*.
Si le mobile est en .NET MAUI, il peut référencer directement la bibliothèque du moteur.	Ce partage n'existe pas pour un mobile en React Native, Flutter ou natif. *Mesure :* le choix du mobile reste guidé par l'équipe mobile et l'accessibilité, l'API suffit.

08 Architecture cible de la solution

```
src/
  Rules/           moteur, dés, effets, ligne de vue (sans infrastructure)
  Rules.Data/      format des effets, sérialisation, JSON Schema
  Srd.Importer/    import rejouable et versionné du SRD 5e (CC BY 4.0)
  Infrastructure/  EF Core, PostgreSQL, migrations
  Api/             endpoints, hubs SignalR, projections par audience
tests/
  Rules.Tests/     personnages de référence, dés, montée de niveau, vision
  Api.Tests/       filtrage de la vue joueurs, permissions, annulation
```

Principes structurants, indépendants du langage mais facilités par .NET :

- **Combat par commandes.** Chaque action produit un résultat proposé, validé ou modifié par le MJ, puis un événement ajouté au journal. L'annulation se fait par événement inverse ou par rejeu (flux A1 de l'annexe).
- **Projection par audience.** L'état complet n'est jamais sérialisé vers la vue joueurs : une fonction dédiée construit explicitement ce qui est visible, et c'est elle que couvrent les tests de filtrage.
- **Ligne de vue.** Shadowcasting sur la grille, murs et portes modélisés comme des segments ; résultat mis en cache par position et invalidé au déplacement, pour mesurer et documenter les performances demandées.

09 Risques et critères de réévaluation

Risque	Probabilité	Mesure
Le contrat API / SignalR dérive entre serveur et clients	Moyenne	Clients générés depuis OpenAPI dans la CI ; contrat de /evaluate et des messages figé dès les premières semaines.
Latence perçue dans l'assistant de création	Faible	Endpoint sans état ; mesure des temps de réponse documentée avec les autres mesures de performance.
Un coéquipier peu à l'aise en C# doit expliquer le code serveur	Moyenne	Revue systématique en pull request, guide de prise en main du moteur, code idiomatique sans abstraction superflue.
Hébergement gratuit peu adapté à .NET	Faible	Déploiement par conteneur Docker, accepté par les offres gratuites courantes ; choix documenté dans un ADR dédié.

La décision sera réexaminée si :

- un mode hors ligne devient nécessaire côté mobile, ce qui imposerait d'exécuter le moteur sur l'appareil ;
- la composition de l'équipe change et le serveur est repris par un membre expérimenté en TypeScript avec un front React et un mobile React Native ;
- les mesures de performance montrent que la synchronisation ou la ligne de vue ne tiennent pas avec une carte chargée.

Sources

- Sujet EpiQuest, Programme Grande École 5e année, Epitech Paris, 2026-2027 (blocs 01 à 05).
- Microsoft, *.NET and .NET Core Support Policy* : `dotnet.microsoft.com/platform/support/policy/dotnet-core` (dates de support de .NET 8, 9 et 10, consultées le 28 octobre 2026).